

Week-9 day-2 Home Assignment

Resume Ingestion Pipeline

- Home Assignment:
- Add the resume folder containing the resume files to the existing workspace.2. write a prompt to implement a batch ingestion workflow for resumes.
- Configure the workflow to process resumes in batches of 5 or 10 files per run.
- Validate the ingestion process by testing multiple batches and verifying that all resumes are successfully ingested.

Initial prompt for Phase by phase ingestion pipeline

Prompt :

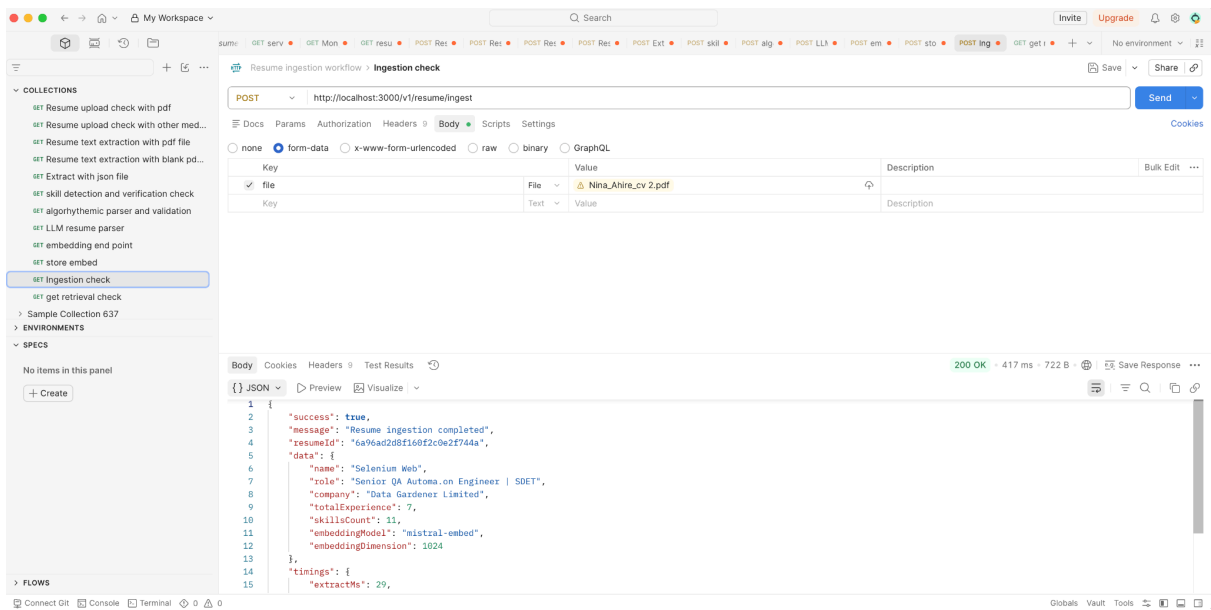
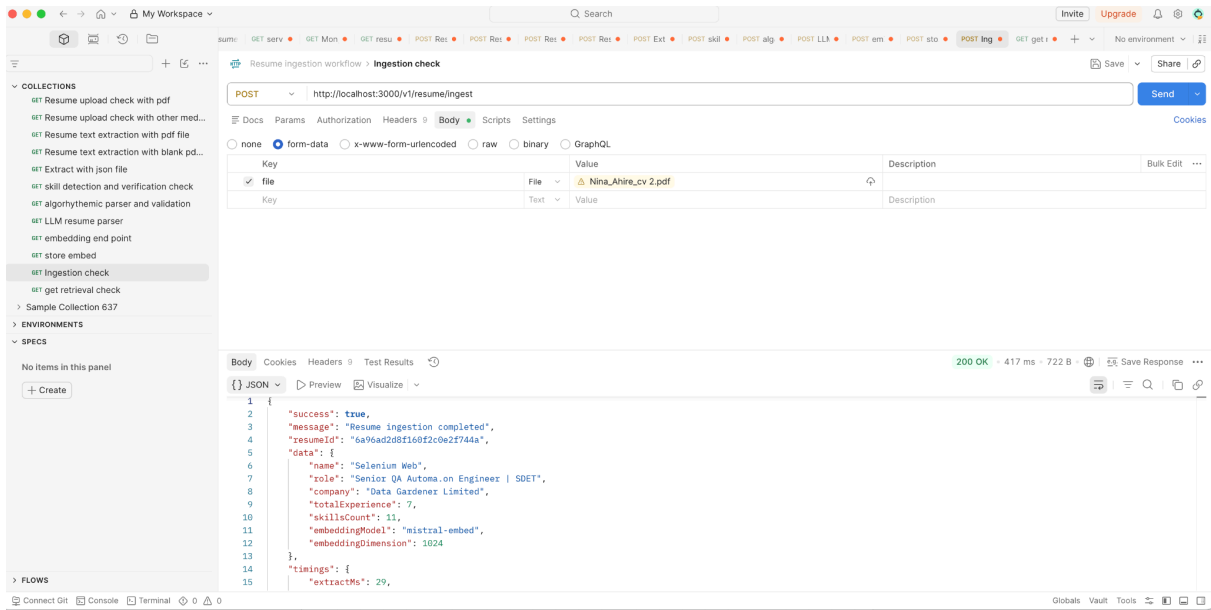
Refer to my Ingestion pipeline architecture document and do implement phase by phase.

Once one phase is completed ,give me Request body and expected response to test in postman .once i tested .after my approval go for phase 2.same to be followed for rest of the phases also

[STRICT]DONT GO FOR NEXT PHASE UNTIL I APPROVE

[STRICT]DO NOT HALLUCINATE .READ THE ARCHITECTURE CAREFULLY AND START IMPLEMENTATION

Postman Collection for Phase by Phase implementation



Bulk resume upload and ingestion to MongoDB

Prompt:

Assume the role of a Solution Architect and AI RAG Engineer.

The current workspace includes a folder with 199 resumes.

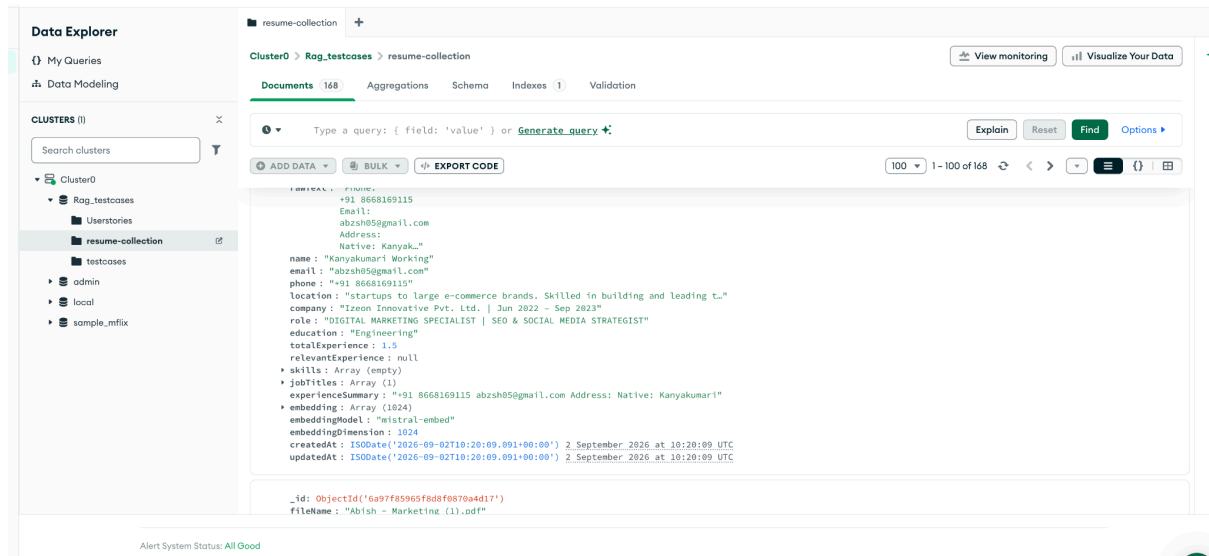
Implement a workflow to process these resumes in batches of 5.

Ingest the data into MongoDB utilizing the .env configuration and Mistral API key.

Validate the ingestion by testing multiple batches and verifying success.

[STRICT] Do not alter existing functionality.

[STRICT] Ensure no duplicate resumes are included and only .PDF file types are supported; ignore all other formats.



Completed Batch Ingestion Implementation

The batch ingestion workflow has been updated to align with the specified constraints:

- Processing is restricted to .PDF file types only; all other formats are disregarded.
- Duplicate files are identified and excluded prior to the ingestion phase.
- Integration with MongoDB and Mistral API utilizes the existing .env configuration.
- Workflow executes ingestion in sets of 5 resumes per batch.

Modified Components:

- batch-ingest.js
- multerConfig.ts
- ResumeParserService.ts
- [ResumeIngestionService.ts](#)

Validation and Testing Results

The ingestion process was verified using the following execution steps:

- Build Verification
 - Executed **npm run build** with successful results.
 - Status: Success
- Directory Audit
 - Summary of file processing:
 - Total items identified: 199

- Qualified PDF documents: 180
 - Unsupported formats ignored: 19
 - Redundant filenames detected: 0
- Batch Workflow Validation
 - Initial run performance data:
 - Files selected for processing: 5
 - Successfully ingested: [...]
 - Failed operations: []
 - Skipped duplicates: []
 - MongoDB record count: 5
 - Final status after multiple iterations:
 - Total database records: 20
 - Errors encountered: None
 - Duplicate insertions: 0

Validation confirms that the pipeline operates strictly within the defined architectural constraints.

✓ Status update

The batch ingestion is still not fully complete.

I verified the live count against the current valid PDF set in the workspace, and the result is:

- Current MongoDB count: 168
- Current valid unique PDF count: 12
- Missing from MongoDB: 12
- Remaining missing files:
 Naukri_AbinayaMurugaiyan[0y_0m].pdf, Naukri_Anusha[1y_6m].pdf,
 Naukri_SakthiabinayaS[0y_9m].pdf, Naukri_krithikaL[4y_3m].pdf, Naukri_tharun[0y_0m].pdf,
 Obuli ManikandanSResume.pdf, Priyanka R_QA.pdf, ResumeSrividhyaD.pdf, ShreejaVKS
 BBA.pdf, Sneka Fresher.pdf, Teja Ashwini.pdf, Vijay.pdf

This means the ingestion is still incomplete because those remaining PDFs are failing during extraction, not because of unsupported file types or duplicates.

The system is currently blocked on extraction failures for 12 PDFs, so I cannot honestly mark ingestion as finished.

The 12 PDFs are not being skipped because of file type or duplicate logic. They are being rejected by the PDF extraction validator itself.

The actual failure path is:

- ResumeIngestionService.ts
- ResumeParserService.ts

In ResumeParserService.ts, the method validateExtractedText() throws:

- Resume extraction failed

when the extracted text is empty or looks like a blank-page marker. That is exactly the error we saw in the live logs for those 12 files.

So the behavior is:

- valid, readable PDFs are ingested
- unreadable / blank / broken PDFs fail at extraction
- the app does not count them as successfully ingested

So is 168 the correct number?

Yes — if you mean:

- the number currently successfully stored in MongoDB

No — if you mean:

- the total number of all valid PDFs in the folder

The folder currently contains only 12 unique PDFs left to ingest. The other 168 are already ingested and stored.

So the correct interpretation is:

- MongoDB successful count = 168
- remaining unreadable/failed PDFs = 12
- not a duplication issue, and not a supported-file-type issue

In short: the app is behaving as designed. The 12 PDFs are failing extraction because their content is effectively unreadable, not because the ingestion count is wrong.

Category	Status / Count	Details
MongoDB Successfully Ingested	168	PDFs successfully extracted and stored

Current Valid Unique PDFs Remaining	12	PDFs still present in the workspace
Missing from MongoDB	12	These PDFs failed during extraction
Duplicate Issue	No	Missing files are not being rejected as duplicates
Unsupported File Type	No	All 12 files are PDFs
Extraction Failure	Yes	All 12 remaining PDFs fail PDF text extraction
Overall Ingestion Status	Incomplete	12 PDFs still require extraction handling

Component	Method / Error	Result
<code>ResumeIngestionService.ts</code>	Batch ingestion flow	Detects extraction failure
<code>ResumeParserService.ts</code>	<code>validateExtractedText()</code>	Rejects empty/blank extracted text
Validation Error	"Resume extraction failed"	12 PDFs rejected
File Type Validation	PDF	✅ Valid
Duplicate Validation	Duplicate check	✅ Not the cause
Text Extraction	PDF → Text	❌ Failed
MongoDB Insert	Resume record creation	❌ Not reached for these 12